

8. Overview and concepts

<  Previous

Next  >



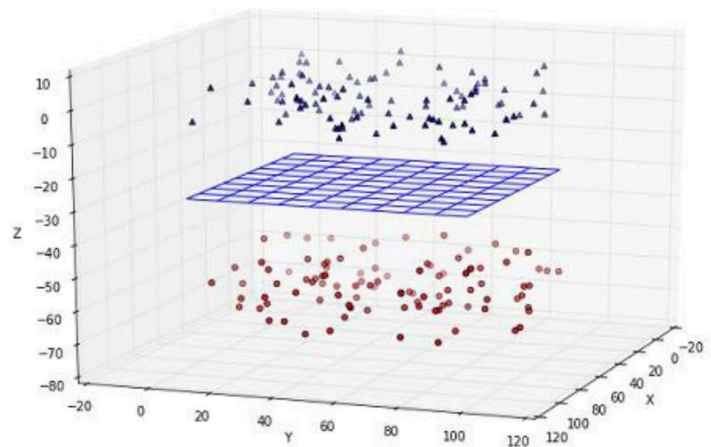
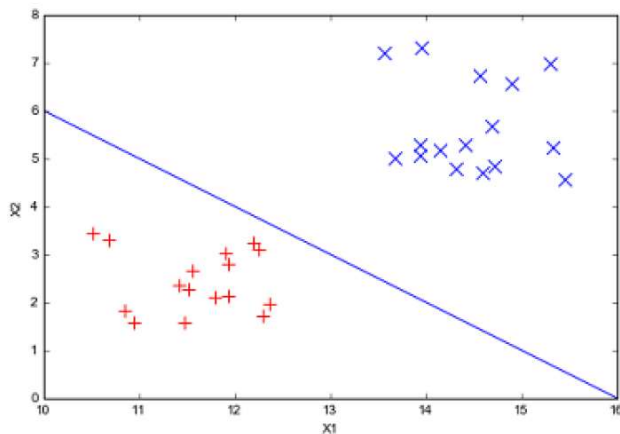
Video: The Concept of SVM

Watch this video to grasp the intuitions behind the Support Vector Machines classifier. Our workshop session will only cover practical activities.

Support Vector Machines (SVMs)

Support Vector Machines is a suite of related classification algorithms that aim to classify data using a *hyperplane* (you will also find out later that a variant of SVMs called SVR is suited for regression).

On the left plot below, the hyperplane is a straight line that separates data points belonging to two different classes, such that any point that is on one side of the hyperplane is classified as belonging to a blue class and anything on the other side as a red class.



Source: Kowalczyk, A. Support Vector Machines Succinctly.

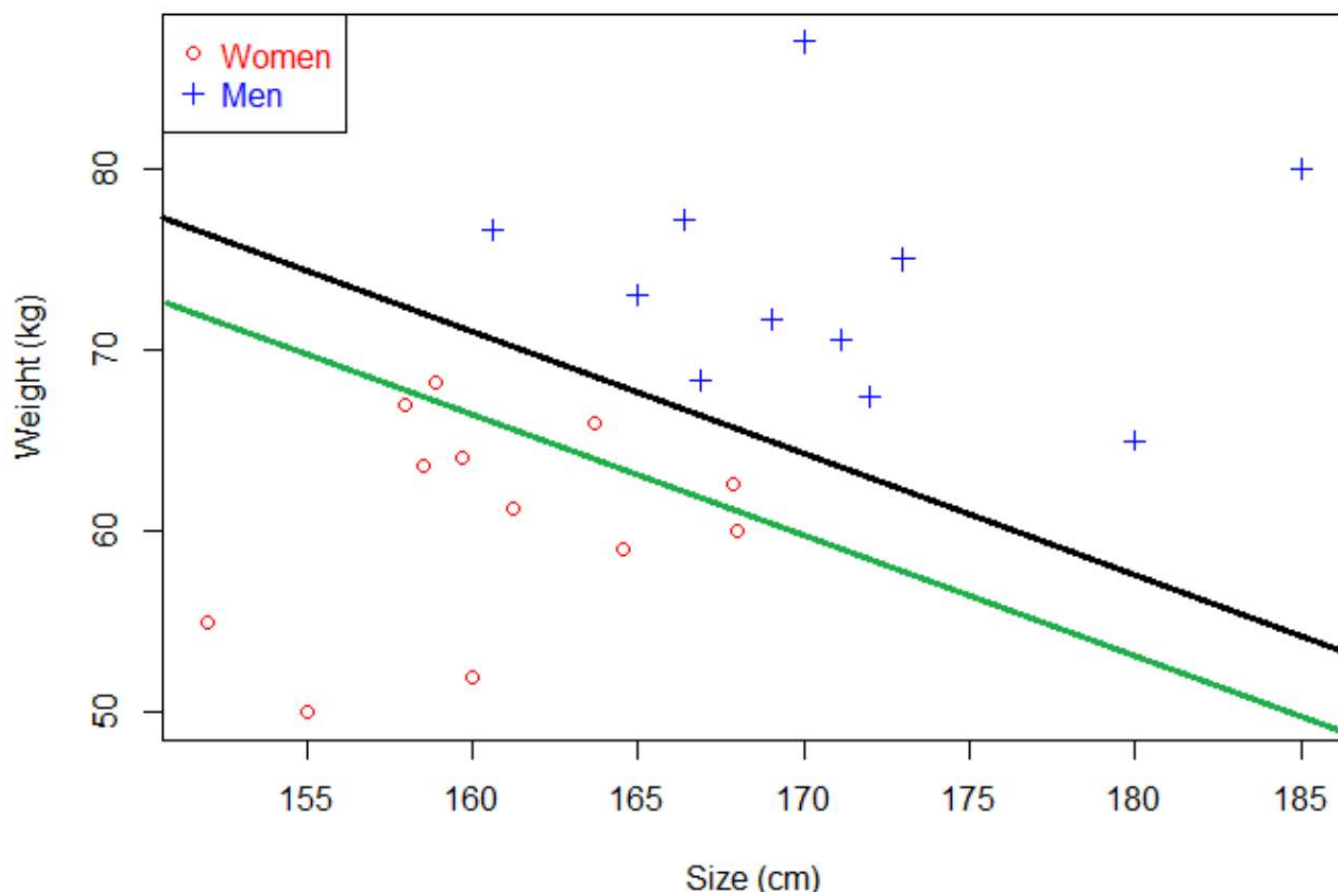
Do not be deterred by the word "hyperplane". A **hyperplane** is just a set of points whose geometric shapes are dictated by the n -dimensional space where it resides. In a 1-dimensional space (i.e. dataset with 1 explanatory variable), a hyperplane is a **single point**. In a 2-dimensional space (2 explanatory variables), it forms a **line**. In a 3-dimensional space, it takes the form of a geometric plane (see the right plot above). For any higher-dimensional space, it becomes difficult to visualise the corresponding hyperplanes. So, we just call them hyperplanes and work with them mathematically. From this point onwards, we explain the SVM model as a line-shaped hyperplane within a 2-dimensional space. Those explanations directly generalise to other higher-dimensional spaces.

Please note that knowing the mathematical details of the algorithm is not essential for using SVM in practice nor is it required by this unit. Before going further, I'd like to acknowledge author Alexandre Kowalczyk and his excellent free e-book [Support Vector Machines Succinctly](https://www.syncfusion.com/succinctly-free-ebooks/support-vector-machines-succinctly) (<https://www.syncfusion.com/succinctly-free-ebooks/support-vector-machines-succinctly>) (highly recommended) from which much of the content here has been adapted. To those interested to know more about the maths behind SVM, please read the book and also visit the blog www.svm-tutorial.com ([https://www.svm-tutorial.com/](http://www.svm-tutorial.com/)) for a quick, nice intro to SVM. In reading those, you need to know basic linear algebra, which is why we covered linear algebra earlier this semester!

Finding the optimal hyperplane

Building an SVM model is about finding the best line (i.e. hyperplane) that separates data points from different classes. It is not difficult to imagine by looking at the previous 2D plot that there are many possibilities for drawing a line that separates the blue and red classes.

For another illustration, observe the following diagram. Both the **black** and **green** lines are hyperplanes, although we can tell that the black one separates the women and men classes more clearly than the green one, according to the weight and size variables.



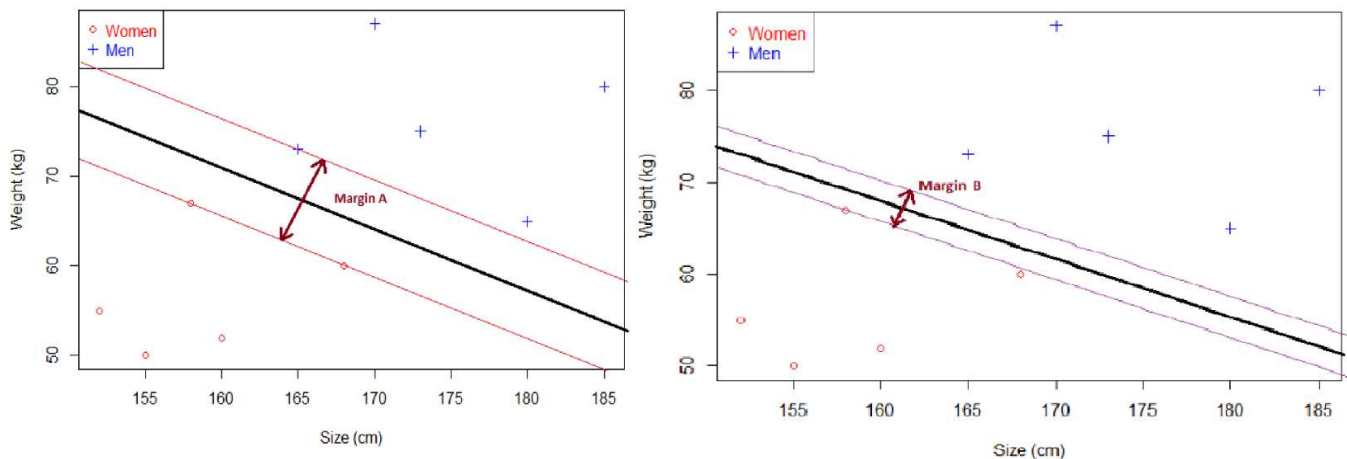
Source: svm-tutorial.com (<https://www.svm-tutorial.com/2014/11/svm-understanding-math-part-1/>).

SVM's way of finding the optimal or the right hyperplane hinges on an important concept called **margin**. Every hyperplane has a *margin*, the distance from the hyperplane to its closest data point (regardless of the class). For instance, in the diagram below, two hyperplanes are plotted against the same set of data points. The first hyperplane on the left plot separates the blue and red classes pretty well with a wide margin (*margin A*). In contrast, the other hyperplane in the next plot seems to fall a little too close to the red data points that its margin (*margin B*) is smaller (the margin is a distance from this hyperplane to its closest data point which happens to be a red one).

In other words:

- if a hyperplane is very close to a data point, its margin will be small.
- the further a hyperplane is from a data point, the larger its margin.

- a margin is determined by the positions of data point(s) closest to a hyperplane. These data points are called the **support vectors** (hence the name SVM).



Source: svm-tutorial.com (<https://www.svm-tutorial.com/2014/11/svm-understanding-math-part-1/>)

Importantly, it has been proven mathematically that **finding the optimal hyperplane is equivalent to finding the one that has the largest margin.**

One may further observe in the plots above that a margin is indeed the distance between a pair of hyperplanes that enclose the primary hyperplane. These are the two red lines that enclose the black one. Consequently, maximising a hyperplane margin is equivalent to maximising the distance between these two supporting red hyperplanes.

In summary, to find the largest margin:

1. We have a dataset.
2. Select two hyperplanes that separate data points with no point between them.
3. Maximize their distance (the margin)

The above summarises the basic intuition behind SVM. Finding the hyperplane with the largest margin is achieved by employing a rather sophisticated optimisation technique that satisfies some nifty mathematical constraints. As mentioned earlier, knowing those maths is not essential to most SVM users and, therefore, is not covered in this unit.

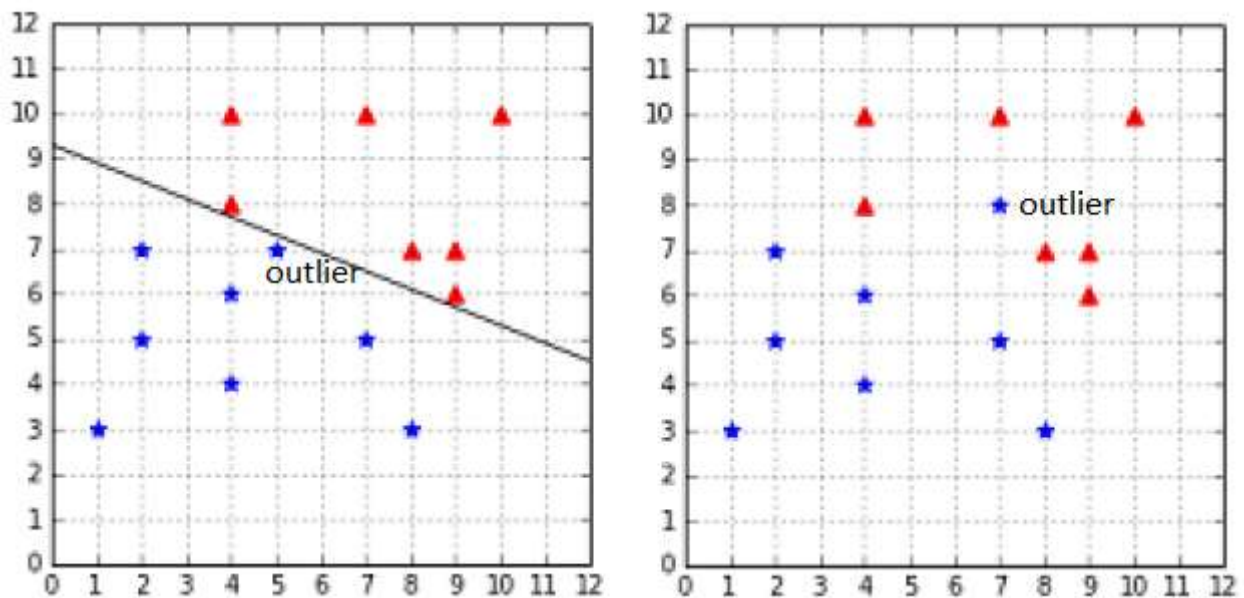
Dealing with noisy data using soft margin SVM

As indicated earlier, finding the right hyperplane for SVM is done by optimising certain mathematical equations while satisfying a set of constraints. At first, these constraints are pretty inflexible, producing the so-called "hard margin" for a hyperplane. This margin works fine if the data points used for training

an SVM classifier are linearly separable and free from unexpected noises. The problem is that many real-world datasets are not always linearly separable and may contain some outliers. In those cases, the hard margin approach may not be as effective.

It is important to understand that, historically, the original SVM optimisation approach was aimed at the total elimination of any classification errors during the training phase. This is what gives it the hard margin. But this assumption proved to be untenable for noisy data. The following plots illustrate the problem. In the left plot, there is an outlier (blue star) near SVM's decision boundary. Although it does not cause the training data to lose its linear separability (i.e. there is still a hyperplane to be found), it causes a hard margin SVM to end up with a hyperplane with a very narrow margin, owing to its effort in eliminating classification errors during training.

Granted that there is no misclassification error during the training phase as shown in the plot. However, this type of hyperplane is prone to producing many misclassification errors on the unseen test set. Slight variations in the values of explanatory variables between the training and the test set could easily confuse the class membership assignment.



Source: Kowalczyk, A. Support Vector Machines Succinctly.

The plot on the right illustrates a different problem. Here, the mere presence of the outlier (a blue star among the red triangles) causes the training data to lose its linear separability. No hyperplane can be found by a hard margin SVM for this case, rendering classification impossible.

In response to these problems, subsequent versions of SVMs were designed not to eliminate errors during the training phase but to *minimise* them. This design principle led to the formulation of "soft margin" SVMs. These are the common versions of SVMs we use nowadays. Having soft margins means SVM incorporates a specific error term into its optimisation's constraint formula when searching for the best hyperplane. In other words, when finding the right hyperplane during the training phase, a certain amount of classification errors is admissible.

Just how much error is admissible?

There is no single answer to that. Instead, researchers introduced an SVM setting or hyperparameter, called *hyperparameter C*, to give SVM users the control over how much classification error is admissible when training the model and how much penalty is applied for each error. This leads us to the concept

called *regularisation*, explained below.

Regularisation and hyperparameter C

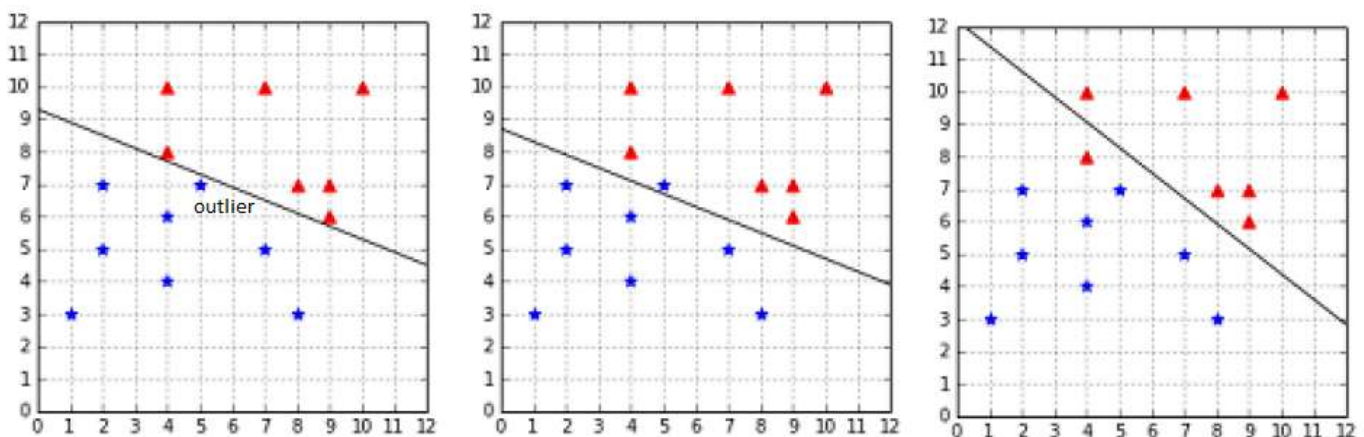
Introducing errors into soft margin SVMs is tied to the idea called **regularisation**. Regularisation provides SVM with the flexibility to relax its hard margin requirement so that it can better handle noisy or non-linearly separable data.

In machine learning, regularisation is a technique that modifies a classifier's function by introducing an additional penalty to its error term. The goal of regularisation is to fit SVM well enough on the training set with some errors, in return for gaining better performance on an unseen test set. This approach avoids *overfitting*, a problem where a classifier seemingly fits excellently on the training set but performs poorly on the test set.

To implement regularisation in SVM, the hyperparameter C was introduced. The values for C have the following regularisation effects:

- A small C assigns a small penalty for misclassification and *softens* the SVM margin. The result is more misclassifications on the training set. However, these are compensated by better generalisation and performance on an unseen test set.
- A large C assigns a large penalty for misclassification and *hardens* the SVM margin. In other words, it becomes more costly for the classifier to make errors during the training phase. As a result, there are fewer misclassifications on the training set. However, this may lead to the overfitting problem and potentially worse performance on an unseen test set.

Consequently, a practical challenge when training an SVM model is to find the optimal value of C with which the classifier performs reasonably well on both the training and test sets. Let's consider some examples. Continuing the previous example of an outlier near the decision boundary, setting different values for hyperparameter C gives the following effects:

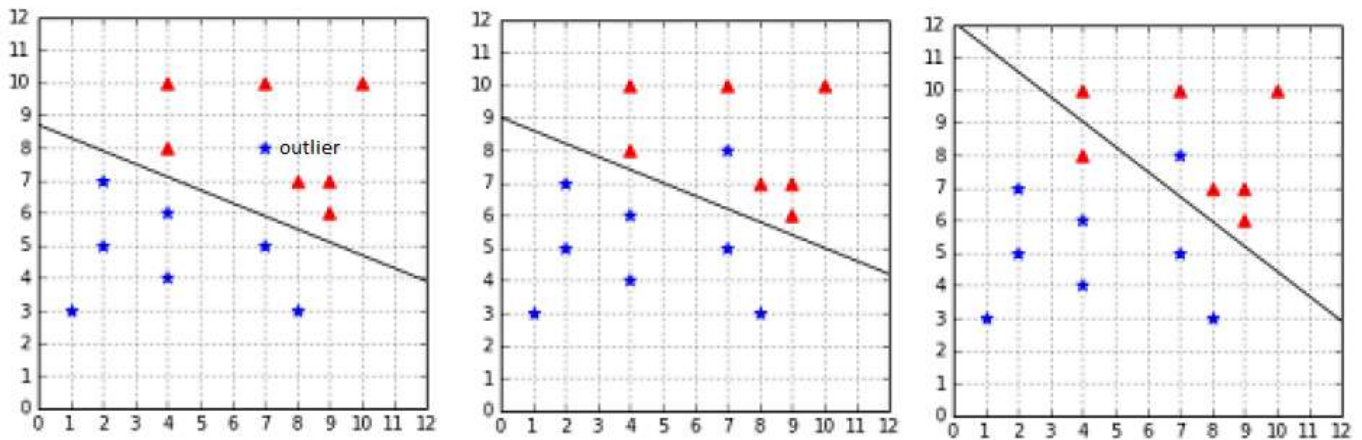


Source: Kowalczyk, A. Support Vector Machines Succinctly.

(from left to right: $C=+\infty$, $C=1$, and $C=0.01$). In the above, setting C to $+\infty$ produces a hyperplane identical to the hyperplane produced by a hard margin SVM. Lowering the value to 1 produces a hyperplane with only one misclassification error during the training phase (i.e. the blue star above the hyperplane), in return for a wider margin hyperplane. Further reducing the value of C

introduces a different classification error (the red triangle below the hyperplane). A pragmatic approach would prefer SVM with $C=1$ over $C=0.01$ because it is more acceptable to misclassify an outlier (the blue star) than to misclassify a non-outlier data point (the red triangle).

For the next problem where an outlier renders the training data to be non-linearly separable, tuning the hyperparameter C has the following effects:



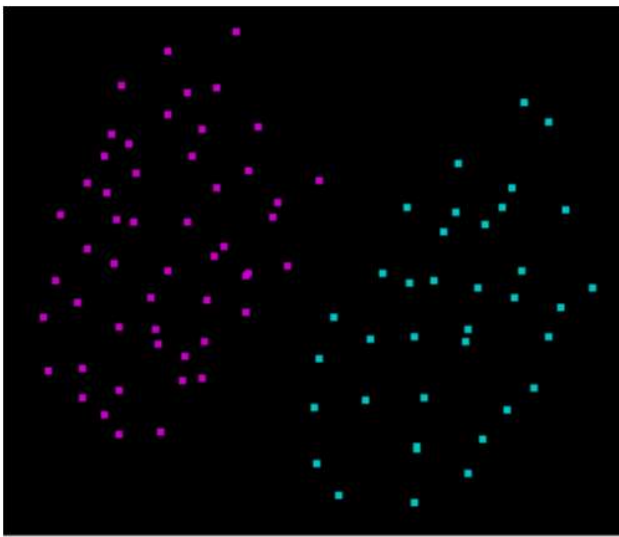
Source: Kowalczyk, A. Support Vector Machines Succinctly.

(from left to right: $C=3$, $C=1$, and $C=0.01$). Here, setting C to either 3 or 1 produces similar hyperplanes because no matter how hard we penalise the errors, the outlying blue star data point always ends up as a misclassification error. We also observe from the rightmost plot that it is not beneficial to further soften the SVM margin by reducing C to 0.01. It only adds additional misclassification errors (the red triangle below the hyperplane) without eliminating the outlier problem. For this situation, sticking to $C=3$ or $C=1$ is likely the most optimal solution.

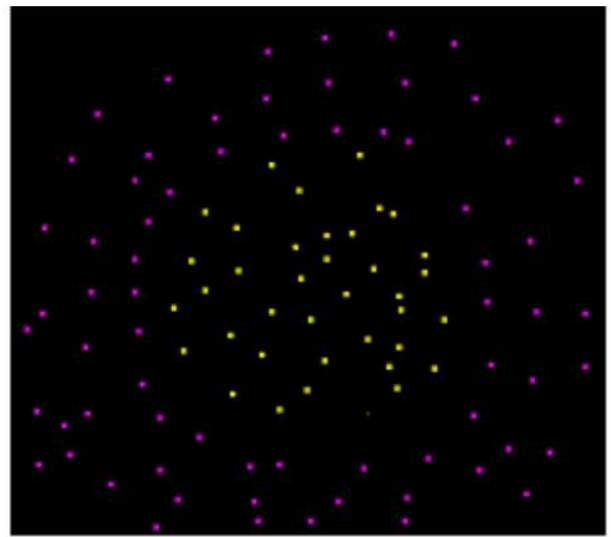
Classifying non-linearly separable data with the kernel trick

As described previously, not all data are linearly separable. This section further deals with this problem.

In the example below, linearly separable data is illustrated on the left. In contrast, non-linearly separable data is shown on the right, for which no straight line exists that cleanly separates the two classes.



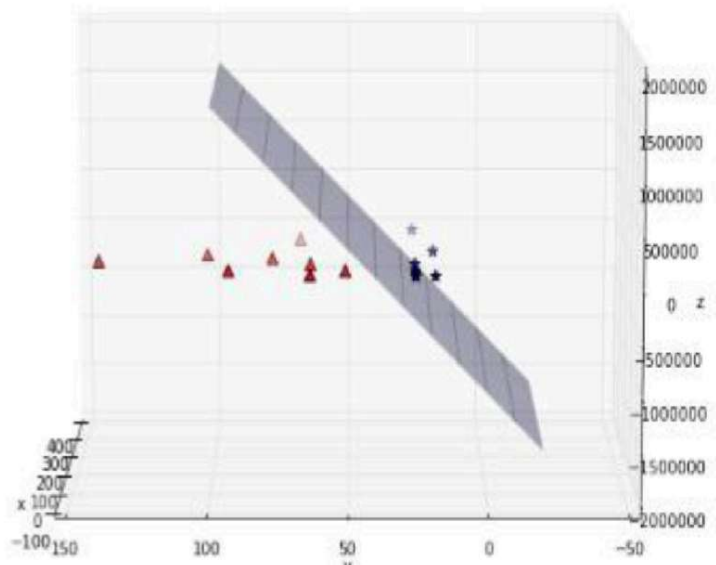
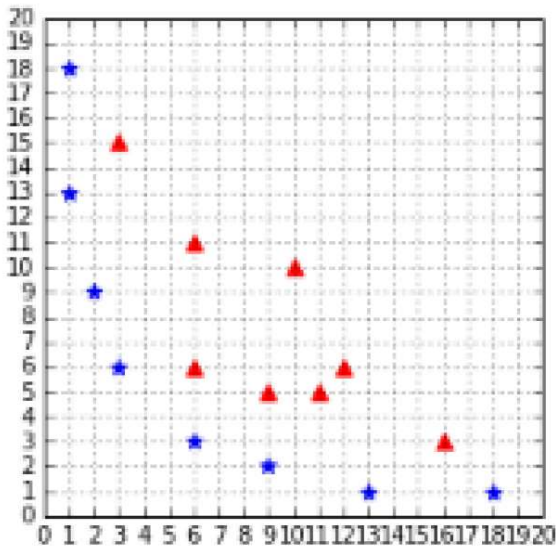
Linearly separable data



Non linearly separable data

Source: svm-tutorial.com (<https://www.svm-tutorial.com/2014/11/svm-understanding-math-part-1/>).

The previous soft margin trick is incapable of dealing with non-linearly separable data. Another trick that helps SVM overcome this problem is to transform such data (by applying a specific transformation formula) to be linearly separable in a higher-dimensional space. For example, the dataset below becomes linearly separable by a hyperplane in a 3D space after being transformed from a 2D space:



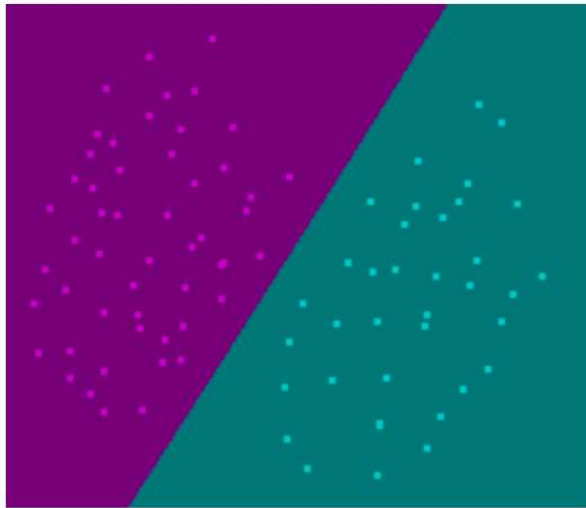
Source: Kowalczyk, A. Support Vector Machines Succinctly.

Unfortunately, such a transformation technique can be computationally expensive. To address this issue, the **kernel trick** was invented for SVM to bypass the transformation process but still achieve the same separation outcome. A kernel is just some mathematical function. Using the right kernel trick depends on choosing the right kernel to use for SVM, explained here.

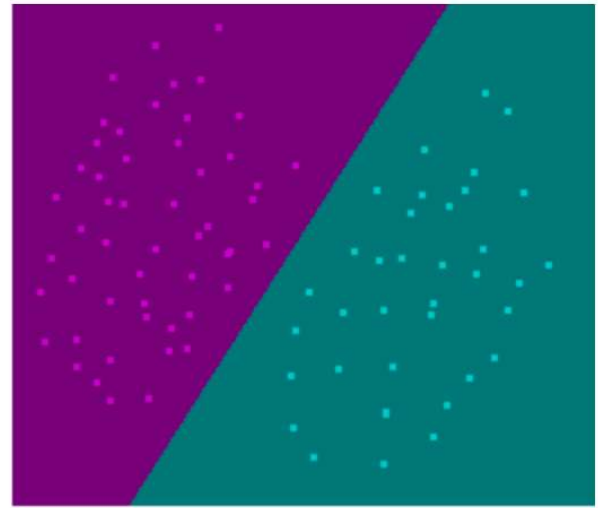
Linear kernel

The linear kernel is used when data is linearly separable. It is the simplest and the fastest kernel in SVM and does not require a lot of hyperparameters to be adjusted to achieve good performance. It is suitable when there are a lot of features in the dataset (such as in text classification tasks) because (i) texts may be linearly separable, and (ii) applying further transformations on data with that many features rarely yields additional performance.

There is little incentive to use other more complicated kernels when the data is linearly separable. In the example below, using the RBF kernel (a sophisticated but slower non-linear kernel) produces the same hyperplane as the linear kernel:



RBF Kernel

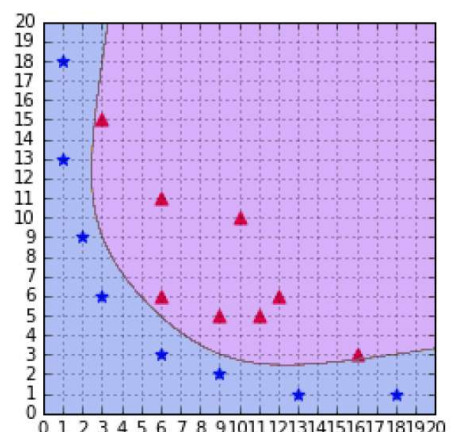
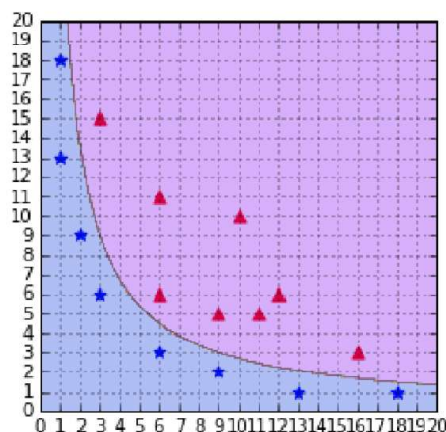
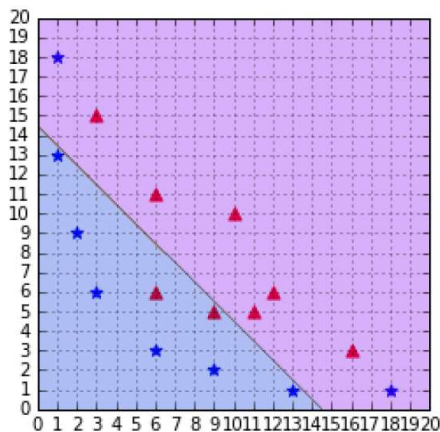


Linear Kernel

Source: svm-tutorial.com (<https://www.svm-tutorial.com/2014/11/svm-understanding-math-part-1/>).

Polynomial kernel

The polynomial kernel is suitable for non-linearly separable data. The decision boundary it produces is controlled by a hyperparameter called the *degree* of the kernel. Setting different degrees changes the decision boundaries of SVM. This diagram shows, from left to right, various decision boundaries by the polynomial kernel with degree=1 (identical to linear kernel), degree=2, and degree=6.



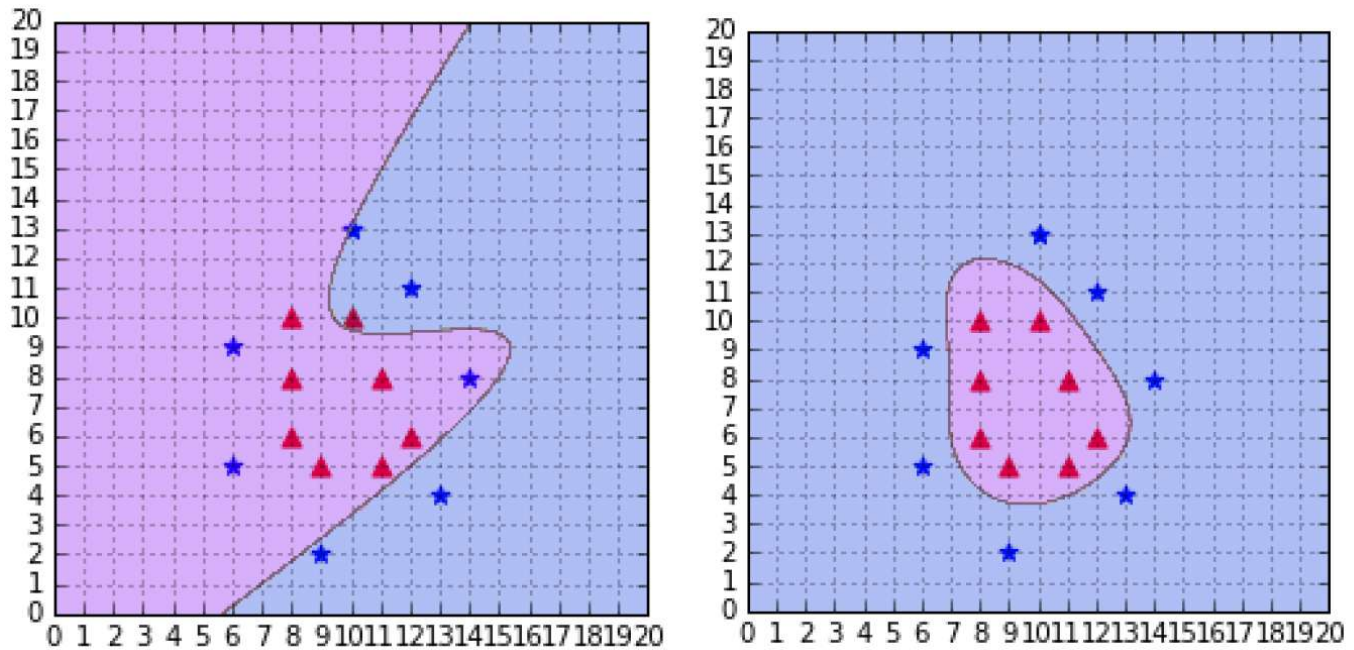
Source: Kowalczyk, A. Support Vector Machines Succinctly.

(from left to right: degree=1, degree=2, degree=6). For the left plot, setting degree=1 is not optimal because the dataset is non-linearly separable and the polynomial kernel produces a similar decision boundary as that of a linear kernel. For the right plot, setting the degree too high (e.g. 6) causes the

decision boundary to overfit the training data so that it may not generalise well to the unseen test set. Finally, in the centre plot, we see that setting degree=2 seems to be an optimal choice as it clearly separates the two classes without much overfitting.

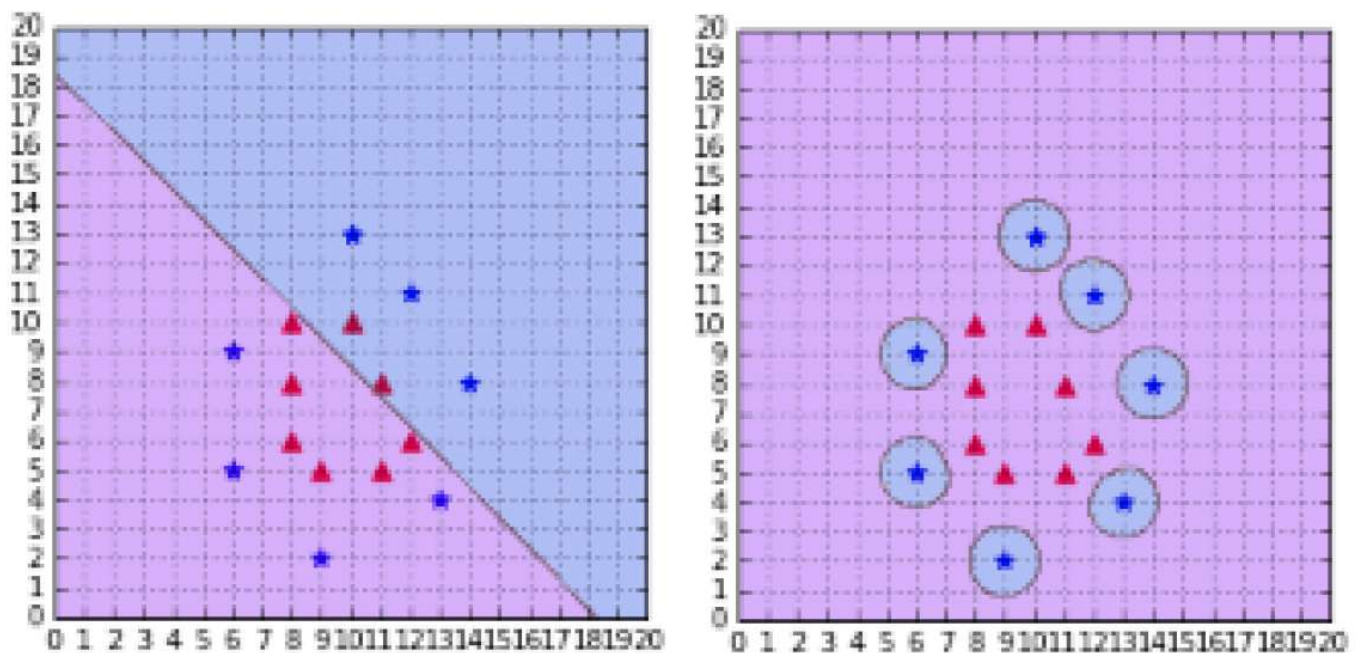
RBF (Radial Basis Function) kernel

There are cases where the polynomial kernel is not good enough, as shown in the left plot below. The RBF kernel can be used to construct a more fitting non-linear decision boundary for SVM (see the plot on the right).



Source: Kowalczyk, A. Support Vector Machines Succinctly.

The behaviour of the RBF kernel, also called the Gaussian kernel, is dictated by a specific hyperparameter called *gamma*. Akin to the polynomial kernel, finding the right gamma value is a matter of trial and error. Choosing a gamma too small produces linear decision boundaries (left plot below). If the gamma is too large, the kernel will overfit the training set (right plot).



Source: Kowalczyk, A. Support Vector Machines Succinctly.

The RBF kernel is the most powerful of all standard SVM kernels. It is a usual practice to use the RBF kernel *first*. This is also why the SVM implementation by the `scikit-learn` has the RBF kernel as the default kernel choice. There are cases where other kernels (linear, polynomial, or others) may perform better than RBF, depending on the nature of the target dataset. A systematic way of searching the right kernel and its appropriate hyperparameter settings (e.g. the grid search) is highly recommended for most modelling scenarios that involve SVM.

Applications

SVM gained its initial popularity in the early 2000s for a handwriting recognition task, where it performed comparably well against the more established neural network algorithm. Subsequently, it became a popular algorithm of choice not just for any general classification but also for document and text classification and image recognition tasks. It is one of the most well-used off-the-shelf supervised machine learning algorithms, often yielding good classification results without complicated tweaks.

SVM has a well-proven mathematical foundation and has been empirically shown to be a reliable classifier. It is also well-known as an effective binary classifier (i.e. it works well for datasets with two classes). It is simple to train and relatively faster than more complex algorithms such as neural networks.

A variant of SVM, Support Vector Regression, is invented to solve regression tasks (i.e. predicting a continuous output variable).

On the other hand, SVM's performance may be limited by the problem of choosing the right kernel function and, depending on the specific SVM libraries you use, may not scale well against extensive, high-dimensional data. It may also not work well when the number of variables (columns) in the dataset

exceeds the number of records (rows).

Finally, an SVM model that relies on default hyperparameter settings does not necessarily give the most optimal performance. Consequently, hyperparameter optimisation is usually needed when using SVM in practice (unlike Naive Bayes which requires little to no hyperparameter optimisation).